

Módulo 7 — Entrando no Protheus: arquitetura e dicionário de dados

Jornada DEV START · Aula 7 · conclusão em 27/07 (segunda) · Programa START — TOTVS Paulista Material entregue ao final da Aula 7. Guarde para consultar sempre que precisar.

 **Nota sobre esta aula:** a turma pediu para ver o Configurador na prática, e a aula tomou um rumo ainda melhor do que o planejado — ao vivo, criamos um **campo customizado**, **renomeamos um campo padrão** e desenhamos **do zero uma tabela nova** (a **ZA1**, de Pets). Por isso este material foi ajustado para refletir **exatamente o que rolou em aula**. O primeiro programa ADVPL (que estava previsto para hoje) fica para o **Módulo 8** — faz mais sentido codar depois de ter passado pela experiência completa do dicionário.

Onde você está na jornada

```
Fundamentos (Harbour)          Protheus (ADVPL)
1 → 2 → 3 → 4 → 5 → 6  ⇒  [7] → 8 → 9 → 10
                        ▲
                    você está aqui — 1ª aula de Protheus! 🎉
```

Grande virada de chave!  Durante seis aulas você aprendeu a **pensar como programador** com Harbour: variáveis, decisões, loops, funções e arrays. A partir de agora, esse mesmo conhecimento vai ser aplicado **dentro do ERP Protheus** — o sistema de gestão da TOTVS que roda em milhares de empresas brasileiras.

Hoje **ainda não** é dia de sair codando (isso começa no Módulo 8). Hoje você **entendeu o terreno**: como o Protheus é montado por dentro, quais ferramentas usa e — o conceito mais importante de toda a parte Protheus — o **dicionário de dados**. E fez isso **na prática**, até criando uma tabela nova do zero.



O que rolou na aula (2h)

Bloco	O quê
Retomada	Fecho dos fundamentos (funções/arrays) + a ponte Harbour → ADVPL
Arquitetura	Server + SmartClient + programa inicial (SIGAMDI); cadastro de cliente ao vivo
Dicionário na prática	MPSDU, índices, Configurador — campo obrigatório, campo novo, aba nova
Desafio da aula	Criação da tabela ZA1 (Pets) do zero, direto no Configurador



O roteiro mudou de ordem em relação ao que outras turmas viram — e está tudo bem. O importante é o **conceito**, não a ordem dos slides.



Objetivos do módulo

Ao final desta aula você é capaz de: - Entender a **arquitetura cliente-servidor** do Protheus: Server + SmartClient + programa inicial - Cadastrar um registro pela tela e **rastrear onde ele foi gravado** (tabela física, DBF) - Usar o **MPSDU** para inspecionar a estrutura e os dados de uma tabela - Explicar para que servem **índices** (analogia da lista telefônica) - Alterar propriedades de campo no **Configurador (SIGACFG)**: tornar obrigatório, renomear título - Criar **campos customizados** e **abas novas** numa tabela padrão (SA1) - Explicar os arquivos **SX2** e **SX3** e o que é obrigatoriedade/internacionalização de campo - Entender **controle de permissões por campo/usuário sem escrever código** - Criar uma **tabela customizada do zero** (convenção de prefixo **Z**) e validar sua persistência

7.1 Retomada: da lógica Harbour para o mundo Protheus

Nos módulos 1–6 você construiu a base: escreveu funções, manipulou arrays, usou **IF**, **DO CASE**, **FOR** e **WHILE**. Tudo isso **continua valendo** — ADVPL e Harbour são mais de 90% compatíveis.

O que muda a partir de agora não é a *lógica*, é o **ambiente onde o código roda** — e é justamente esse ambiente que exploramos hoje, antes de escrever a primeira linha de ADVPL

(isso fica para o Módulo 8):

Antes (Harbour)	Agora (ADVPL / Protheus)
Você escrevia <code>.prg</code> no VSCode	Você escreve <code>.prw</code> no DevStudio/TDS
Compilava com <code>hbmk2</code> e gerava um <code>.exe</code>	Compila para dentro de um repositório chamado RPO
Rodava no terminal da sua máquina	Roda dentro do SmartClient , conversando com um servidor
Você controlava tudo (do zero)	Você estende um ERP que já existe

💡 **Ideia-chave da retomada:** você não vai jogar fora o que aprendeu. Vai **encaixar** esse conhecimento numa plataforma que já traz banco de dados, telas e regras prontas.

7.2 Arquitetura: Server, SmartClient e o programa inicial

Todo acesso ao Protheus passa por três peças:

- **Server (AppServer)** — o servidor que processa o código e conversa com o banco de dados.
- **SmartClient** — a interface gráfica que o usuário abre na própria máquina.
- **Programa inicial (SIGAMDI)** — o “hall de entrada”: a primeira tela que aparece ao logar, de onde você escolhe o módulo que vai usar.

Na prática, existem dois jeitos de chegar num módulo como o **Faturamento**:

1. **Direto** — o SmartClient já abre configurado para entrar direto num módulo específico.
2. **Pelo menu inicial (SIGAMDI)** — você loga, cai numa tela de seleção e escolhe o módulo.

🏢 **Por que isso importa em ambiente corporativo?** Porque é esse menu inicial que permite controlar **quem acessa o quê**. Uma empresa pode liberar o Faturamento para o time comercial e esconder o Financeiro — a escolha do módulo (direto ou pelo menu) é também uma **camada de permissão**.

7.3 Cadastro ao vivo: criando um cliente no Faturamento

Para sentir o Protheus “por dentro”, cadastramos um cliente novo ao vivo no módulo de **Faturamento**, usando como exemplo o cliente “**Michael Jackson**”.

Pontos que apareceram no cadastro:

- Existem **campos obrigatórios** — o Protheus não deixa salvar sem eles (isso vem do dicionário de dados, que vamos ver em detalhe já a seguir).
- Foi comentado que, em **versões antigas** do Protheus, alguns valores (como a **cotação** de moeda) precisavam ser **atualizados manualmente**. Em versões mais novas, isso é resolvido por **jobs em background** — processos automáticos que rodam no servidor sem intervenção do usuário.

💡 Isso é um exemplo de como o Protheus **evolui como plataforma**: tarefas que antes eram manuais vão sendo automatizadas por trás da tela, sem que o desenvolvedor precise “reinventar”.

7.4 Onde o dado foi gravado: MPSDU e o arquivo físico

Depois de cadastrar o cliente, a pergunta natural é: **onde isso foi salvo?**

A resposta: numa **tabela física chamada SA1**, guardada em um **arquivo DBF** no servidor. Para ver isso ao vivo, usamos a ferramenta **MPSDU**, que permite abrir a tabela e localizar o registro recém-criado.

⚠️ **Atenção — isto é importante**: acessar o arquivo direto (como fizemos no MPSDU) é uma prática **só para fins de desenvolvimento e aprendizado**. Em um ambiente de produção real, **nunca** se edita ou consulta dados assim — sempre pela aplicação (SmartClient) ou por rotinas controladas. Acesso direto ao arquivo pode corromper índices, ignorar validações e travar outros usuários.

Índices: a analogia da lista telefônica

Uma tabela com milhares de registros não pode ser “lida do início ao fim” toda vez que alguém busca um cliente — seria lento demais. Para isso existem os **índices**.

 **Analogia:** pense numa lista telefônica antiga. Ela não está em qualquer ordem — está organizada **alfabeticamente**. Isso permite achar um nome rapidamente, sem folhear página por página. Um índice no banco de dados faz o mesmo: organiza os registros por uma chave (ex: código do cliente) para que a busca seja **rápida** e para **evitar duplicidade** (dois registros com a mesma chave).

7.5 Configurador (SIGACFG): mudando propriedades de campo

Voltamos ao **Configurador** — o “painel de controle” do Protheus — para alterar, ao vivo, propriedades de um campo já existente na SA1.

Tornando o campo “bairro” obrigatório e renomeando o título

Na tela de propriedades do campo **bairro** da SA1, fizemos duas mudanças:

1. Marcamos o campo como **obrigatório** — o Protheus passa a **exigir** o preenchimento.
2. Renomeamos o **título** exibido na tela de **"Bairro"** para **"bairrozinho"** — só para deixar visualmente claro, ao vivo, que a mudança realmente “pegou”.

 **Detalhe operacional importante:** alterar a **estrutura** de uma tabela (adicionar campo, mudar obrigatoriedade, etc.) é uma operação que **trava a tabela** por um instante. Por isso, em ambientes reais, esse tipo de atualização deve rodar em **horário de baixa utilização** — de madrugada, por exemplo — para não travar quem está usando o sistema.

Depois da alteração, voltamos ao **SmartClient** e tentamos cadastrar/alterar um cliente sem preencher o bairro: o sistema **passou a exigir o campo**, confirmando que a mudança no dicionário realmente afeta a tela — sem que uma linha de código tenha sido escrita.

 **A grande sacada do Configurador:** você muda uma **regra no dicionário** e a **tela muda sozinha**. Você não programou nada — só configurou.

7.6 Campo customizado e aba nova na SA1

Ainda no Configurador, criamos um **campo customizado** na tabela SA1:

- **Nome do campo:** `A1_V0V0`
- Para organizá-lo na tela, criamos também uma **aba nova**, chamada “**frescura**”.

Depois de criar o campo e a aba, voltamos ao **MPSDU** para confirmar que a estrutura da SA1 realmente ganhou o campo novo.

 **Por que o nome começa com `A1_` ?** Porque `A1` é o prefixo da tabela SA1. Todo campo de uma tabela do Protheus começa com o prefixo da tabela a que pertence — é assim que se sabe, só olhando o nome do campo, de qual tabela ele é.

7.7 Arquivos SX: o motor por trás do dicionário

Os campos e tabelas que você configura no Configurador são, na verdade, **registros dentro de tabelas especiais do próprio Protheus**, chamadas **arquivos SX**:

Arquivo	Para que serve
SX2	Mapeamento das tabelas — quais tabelas existem, nome, caminho do arquivo
SX3	Definição dos campos — tipo, tamanho, título, obrigatoriedade, validação...

Ao explorar o SX3, vimos dois detalhes importantes:

- **Obrigatoriedade** — a mesma propriedade que usamos para o campo bairro é, por baixo dos panos, um atributo guardado no SX3.
- **Internacionalização** — o título de um campo pode ter uma versão em **português**, **espanhol** e **inglês**. O Protheus troca automaticamente o idioma da tela conforme o idioma configurado para o usuário.

 **Curiosidade comentada em aula:** a versão do Protheus usada tem **2.393 tabelas** cadastradas no SX2. E o modelo de licenciamento hoje, diferente de antigamente, é por **assinatura (SaaS)** — a empresa paga um valor recorrente pelo uso, em vez de comprar uma licença única.

7.8 Permissões por campo e usuário — sem código

Outro ponto mostrado no Configurador: é possível controlar **quem pode ver ou editar** cada campo, **por usuário ou por grupo**, sem escrever nenhuma linha de ADVPL. Isso é configuração pura de dicionário + segurança — o mesmo espírito de “mudou a regra, a tela obedece” que vimos no bairro obrigatório.

7.9 Desafio da aula: criando a tabela ZA1 (Pets) do zero

Este foi o ponto alto da aula: em vez de só explorar tabelas que já existem, a turma **criou uma tabela nova do zero**, direto no Configurador.

O desafio

Cadastrar uma tabela para guardar **Pets**, com os campos:

- Nome do pet
- Raça
- Data de nascimento

Por que o nome é ZA1

- O prefixo **Z** é a convenção do Protheus para **tabelas customizadas de cliente** — ou seja, tabelas que **não vêm de fábrica** com o ERP, mas foram criadas para uma necessidade específica (no nosso caso, um exemplo didático de Pets).
- Assim como toda tabela padrão (**SA1** , **SB1** ...), uma tabela customizada também tem um prefixo próprio — escolhemos **ZA1** . Os campos seguem a **mesma regra do SA1**: **começam com o nome da tabela**. Por isso os campos ficaram **ZA1_NOME** , **ZA1_RACA** , **ZA1_NASC** (e não “NOME” solto).



Regra geral (vale para sempre): nome do campo = prefixo da tabela + **_** + nome do campo. Isso é o que faz o Protheus (e qualquer desenvolvedor lendo seu código) saber, só pelo nome, de qual tabela é aquele campo.

Decisões que tomamos ao montar a ZA1

- **Compartilhamento entre empresas** — definimos se a tabela ZA1 seria compartilhada entre empresas diferentes do mesmo ambiente ou exclusiva de uma empresa (o mesmo conceito de compartilhado/exclusivo que existe nas tabelas padrão).

- **Campo de filial obrigatório** — assim como toda tabela do Protheus, a ZA1 **precisa** de um campo de filial (`ZA1_FILIAL`). Sem ele, a tabela não se encaixa no padrão multiempresa do Protheus.
- **Dificuldades reais que apareceram e como resolvemos:**
 - Tivemos dificuldade com o **caminho do arquivo** da nova tabela — resolvido seguindo o **mesmo padrão usado pela SA1**.
 - Também tivemos que ajustar quais campos eram **obrigatórios**, de novo seguindo o padrão da SA1 como referência.
 - Descobrimos, na prática, que existe uma **restrição de tamanho para o título de um campo** quando ele aparece no **Browse** (a listagem) — título grande demais não cabe/não é aceito.

Validando a tabela nova

Depois de cadastrar a estrutura da ZA1 no dicionário, ela ainda não estava “reconhecida” de prática pelo framework. Usamos a **rotina de fórmulas** do Protheus para **forçar esse reconhecimento** — e então conferimos a estrutura final no **MPSDU**, vendo a tabela ZA1 já criada com seus campos.

⚠ **Ficou pendente — e combinado em voz alta:** a ZA1 ainda **não tem nenhuma amarração com o cliente** (ou seja, não sabemos, pela tabela, **de quem** é aquele pet). Isso é proposital: vamos resolver isso **na aula seguinte**, junto com o **início da codificação em ADVPL**. Guarde essa pergunta: “*como eu ligo o pet ao cliente dono dele?*” — ela é o gancho para o Módulo 8.

📌 Referência rápida do Módulo 7

Arquitetura:

```
Server (AppServer) + SmartClient + Programa inicial (SIGAMDI)
                    acesso direto a um módulo OU pelo menu inicial (permissões)
```

Onde o dado mora:

```
Tela (SmartClient) → grava em tabela física (ex: SA1) → arquivo DBF no servidor
Inspeccionar (só em desenvolvimento!): MPSDU
```

Índices: organizam a tabela por uma chave — evitam duplicidade e aceleram a busca (como uma lista telefônica em ordem alfabética).

Configurador (SIGACFG):

Alterar propriedade de campo (obrigatório, título) → atualizar estrutura (horário de baixa utilização)
Criar campo customizado (ex: A1_V0V0) + aba nova (ex: "frescura")
Controlar permissão por campo/usuário — sem escrever código

Arquivos SX:

SX2 → mapeamento das tabelas SX3 → definição dos campos (obrigatoriedade, i18n pt/es/en)

Tabela customizada (convenção):

Prefixo Z → tabela customizada de cliente (ex: ZA1)
Campos → começam com o nome da tabela (ZA1_NOME, ZA1_RACA, ZA1_NASC, ZA1_FILIAL...)
Toda tabela precisa de um campo de FILIAL (multiempresa)

Desafio da aula: tabela **ZA1 — Pets** (nome, raça, data de nascimento), validada no MPSDU via rotina de fórmulas. Pendência combinada: falta a **amarração com o cliente** (dono do pet).



Seu primeiro programa ADVPL — no Módulo 8

Hoje foi só **dicionário de dados e Configurador** — nenhuma linha de ADVPL foi escrita, compilada ou executada em aula, e está tudo bem: era o combinado. O **primeiro programa ADVPL** (a `USER FUNCTION`, o `MsgInfo`, compilar com **F9** e rodar no SmartClient) abre o **Módulo 8**, junto com a resolução da pendência da ZA1 (a amarração com o cliente).



Exercícios

Os exercícios desta aula estão em `exercicios.md` (nesta mesma pasta). Confira lá o prazo de entrega e o aviso sobre o suporte no Discord.

Nesta aula há uma mistura de exercícios **conceituais** (respondidos por escrito, funcionam sem o ambiente) e **práticos** (dependem do ambiente Protheus). O arquivo deixa claro qual é qual.



Para a próxima aula

Módulo 8 — CRUD no Protheus: comece a codificar. Você vai escrever seu **primeiro programa ADVPL** e resolver a pendência que ficou hoje: como **ligar a ZA1 (Pets) ao cliente dono do pet**. A partir daí, o módulo segue para uma tela de cadastro completa (incluir, alterar, excluir, consultar).



Antes da Aula 8: revise os **arquivos SX2/SX3**, a convenção de **prefixo Z** e por que **toda tabela do Protheus precisa de um campo de filial**. Dúvida? Chame no Discord [#duvidas-parte2-protheus](#) .